# Machine Learning — Challenge 3
# Proximal Policy Optimization for Atari (PPO)
# Algorithmic Comparison with Challenge 1

**Prof. Carlos Andrés Sierra, M.Sc.**
Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

## Overview

This document describes Challenge 3 for the Machine Learning course. It is a direct extension of Challenge 1, where each group implemented and tuned a Deep Q-Network (DQN) agent on a specific Atari ALE environment. In Challenge 3, **each group works on the same environment** as in Challenge 1 but now implements a **Proximal Policy Optimization (PPO)** agent, conducts a comparable set of experiments, and produces a side-by-side empirical comparison between the two algorithms.

The central research question this challenge asks students to answer is:

> *"Under a fixed computational budget and on the same environment, does PPO converge faster, reach higher performance, or exhibit different failure modes compared to the DQN agent designed in Challenge 1? Why?"*

## Challenge objective

Groups will:

1. Implement a PPO agent with clipped surrogate objective, Generalised Advantage Estimation (GAE), and shared or separate actor/critic network for the ALE environment assigned in Challenge 1.

2. Conduct a hyperparameter search for the PPO agent analogous to the search performed in Challenge 1, following the same experimental protocol (fixed seeds, repeated runs, logged metrics).

---

Carlos Andrés Sierra, Computer Engineer, M.Sc. in Computer Engineering, Full-time Adjunct Professor at Universidad Distrital Francisco José de Caldas.

Any comment or concern about this document can be sent to: *cavirguezs@udistrital.edu.co*.

3. Directly compare the best PPO run and the best DQN run from Challenge 1 using equivalent metrics: sample efficiency (reward vs. environment steps), final asymptotic performance, and training stability (variance across seeds).

4. Produce a scientific report in IEEE format that extends the Challenge 1 paper with PPO results and a rigorous algorithmic comparison.

**Environments - same assignment as Challenge 1**

Each group works on the **same ALE game** assigned in Challenge 1. The table below is reproduced for reference. Use the same ALE version identifier as before so that DQN and PPO results are directly comparable.

1. Group 1 — **ALE/MontezumaRevenge-v5**: very sparse rewards; long-term planning and room memory required.

2. Group 2 — **ALE/Pitfall-v5**: sparse/negative rewards; delicate action sequences needed to score.

3. Group 3 — **ALE/PrivateEye-v5**: extremely sparse rewards; complex multi-objective structure.

4. Group 4 — **ALE/Gravitar-v5**: challenging physics; precise control under gravity.

5. Group 5 — **ALE/Solaris-v5**: multi-stage dynamics; varied objectives across levels.

6. Group 6 — **ALE/Venture-v5**: sparse positive signals; noisy action space early.

7. Group 7 — **ALE/MsPacman-v5**: dense moving objects; partial observability; reactive planning.

8. Group 8 — **ALE/Phoenix-v5**: fast action dynamics; non-trivial reward structure.

**Preprocessing must be identical to Challenge 1:** grayscale, resize to $84 \times 84$, frame-stack 4, frame-skip 4. Keeping preprocessing constant ensures any observed performance difference is attributable to the algorithm, not to environment configuration.

**PPO algorithm - required elements**

At minimum, the PPO implementation must include:

- On-policy rollout collection for $T$ environment steps (*horizon*).
- Advantage estimation using Generalised Advantage Estimation (GAE) with parameters $\gamma$ and $\lambda$.
- Clipped surrogate objective: $L^{\text{CLIP}} = \mathbb{E}\Big[\min\Big(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}_t\Big)\Big]$ where $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$.
- Value-function loss: $L^{\text{VF}} = \mathbb{E}\big[(V_\theta(s_t) - R_t)^2\big]$.

- Entropy bonus: $L^{\mathrm{ENT}} = \mathbb{E}[H(\pi_\theta(\cdot|s_t))]$ to encourage exploration.
- Combined loss: $L = -L^{\mathrm{CLIP}} + c_1 L^{\mathrm{VF}} - c_2 L^{\mathrm{ENT}}$.
- Multiple mini-batch epochs per horizon (typically $K = 4$–$10$).
- Optional (but encouraged): gradient norm clipping; separate actor and critic networks or a shared convolutional backbone with dual heads.

Groups may use the reference PPO implementation discussed in course lectures as a starting point but must adapt and instrument it for the Atari observation space (convolutional feature extractor, discrete action head).

**Suggested hyperparameter search space for PPO**

- Learning rate: $1 \times 10^{-4}$, $2.5 \times 10^{-4}$, $5 \times 10^{-4}$.
- Horizon ($T$ steps per rollout): 512; 1 024; 2 048.
- Number of epochs per horizon ($K$): 4; 6; 10.
- Mini-batch size: 64; 128; 256.
- Discount factor $\gamma$: 0.99; 0.995.
- GAE $\lambda$: 0.90; 0.95; 0.97.
- Clip epsilon $\varepsilon$: 0.1; 0.2.
- Entropy coefficient $c_2$: 0.001; 0.01; 0.02.
- Value-loss coefficient $c_1$: 0.5; 1.0.
- Gradient norm clip: 0.5; 1.0; none.

Apply the same exploration strategy chosen in Challenge 1 (or justify a different one) and ensure each condition is repeated for at least three independent random seeds.

**Comparison methodology**

The following protocol must be followed so that the DQN vs. PPO comparison is fair:

1. **Budget parity**: define a fixed number of environment steps (e.g., 5 000 000) and train both agents to that exact budget. Report learning curves sampled at the same step intervals.
2. **Identical preprocessing**: same grayscale/resize/stack/skip as Challenge 1.
3. **Identical evaluation protocol**: at fixed intervals, pause training and run 10 deterministic evaluation episodes (epsilon = 0 for DQN; greedy policy for PPO). Report mean and standard deviation of episode return.

4. **Metrics to report**:
   - Learning curve: episode return vs. environment steps.
   - Sample efficiency: steps needed to reach a target score threshold (choose a meaningful threshold, e.g. 50 or 100 points above baseline).
   - Final performance: mean ± std over 3 seeds at the end of training.
   - Training stability: area under the learning curve (AUC) normalised by total steps.
   - Wall-clock time (if compute conditions are comparable).
5. **Analysis**: discuss which algorithm converges faster, which is more stable, and which achieves higher asymptotic performance. Explain these differences in terms of the algorithmic properties of DQN (off-policy, replay buffer, epsilon-greedy) vs. PPO (on-policy, rollouts, clipped ratio, entropy).

## Deliverables

- **Repository folder:** Add a challenge3/group<k>/ folder to the same GitHub repository used for Challenge 1. Include all PPO source code, a README.md with reproducible run instructions, and logging artifacts.
- **Extended IEEE paper:** An 8-page IEEE conference paper extending the Challenge 1 paper. The paper must include all DQN results from Challenge 1 and the new PPO results, a comparison section, and updated conclusions. Submit as challenge3_group<k>_paper.pdf.
- **Checklist:** A short CHECKLIST.md inside the repository folder containing the exact training commands for the best PPO run, seed values, pointers to logs/figures, and a 200-word comparative summary of DQN vs PPO.

## Evaluation criteria

- **Implementation correctness (25%):** PPO algorithm is correctly implemented (clipped objective, GAE, entropy bonus, mini-batch updates).
- **Experimental rigour (30%):** hyperparameter search is systematic, variance is reported, runs are reproducible.
- **Comparison quality (30%):** fair protocol, meaningful metrics, and insightful analysis that links algorithmic properties to observed behaviour.
- **Presentation and writing (15%):** quality of the IEEE extension (clarity, figures, tables, citations, and updated conclusions).

### Per-game starter guides

The following skeletons share a common PPO loop structure. Each snippet highlights the environment setup and per-game-specific considerations students should investigate. The shared preprocessing wrapper shown below applies to **all** groups:

Listing 1: Shared preprocessing wrapper (all groups)

```python
import gymnasium as gym
import numpy as np
from gymnasium.wrappers import (
    GrayscaleObservation,
    ResizeObservation,
    FrameStackObservation,
    AtariPreprocessing,
)

def make_env(env_id: str, seed: int = 0):
    """Build a pre-processed \texttt{ALE} environment compatible with \texttt{PPO}."""
    env = gym.make(env_id, render_mode=None)
    env = AtariPreprocessing(
        env,
        noop_max=30,
        frame_skip=4,
        screen_size=84,
        grayscale_obs=True,
        scale_obs=True,       # pixel values in [0, 1]
        grayscale_newaxis=True,
    )
    env = FrameStackObservation(env, 4)   # stack 4 frames
    env.reset(seed=seed)
    return env
```

Listing 2: Shared convolutional actor-critic backbone (all groups)

```python
import torch
import torch.nn as nn

class AtariActorCritic(nn.Module):
    """Shared CNN backbone with separate actor and critic heads."""

    def __init__(self, n_actions: int):
        super().__init__()
        # Input: (batch, 4, 84, 84) - 4 stacked greyscale frames
        self.cnn = nn.Sequential(
            nn.Conv2d(4, 32, kernel_size=8, stride=4), nn.ReLU(),
            nn.Conv2d(32, 64, kernel_size=4, stride=2), nn.ReLU(),
            nn.Conv2d(64, 64, kernel_size=3, stride=1), nn.ReLU(),
            nn.Flatten(),
        )
        cnn_out = 64 * 7 * 7  # 3136

        self.actor = nn.Sequential(
            nn.Linear(cnn_out, 512), nn.ReLU(),
            nn.Linear(512, n_actions),
```

```
        )
        self.critic = nn.Sequential(
            nn.Linear(cnn_out, 512), nn.ReLU(),
            nn.Linear(512, 1),
        )

    def forward(self, x):
        # x: (batch, 4, 84, 84), dtype float32
        feats = self.cnn(x)
        return self.actor(feats), self.critic(feats).squeeze(-1)
```

Listing 3: Minimal PPO training loop (all groups - adapt as needed)

```
import torch
import torch.optim as optim
from torch.distributions import Categorical

def train_ppo(env_id, total_steps=5_000_000, horizon=1024,
              n_epochs=4, batch_size=128, lr=2.5e-4,
              gamma=0.99, gae_lambda=0.95,
              clip_eps=0.2, ent_coef=0.01, vf_coef=0.5,
              max_grad_norm=0.5, seed=42):

    env = make_env(env_id, seed=seed)
    n_actions = env.action_space.n
    model = AtariActorCritic(n_actions).to("cuda" if torch.cuda.
        is_available() else "cpu")
    optimizer = optim.Adam(model.parameters(), lr=lr)

    obs, _ = env.reset()
    episode_return, all_returns = 0.0, []

    for global_step in range(0, total_steps, horizon):
        # --- rollout collection ---
        obs_buf, act_buf, logp_buf, rew_buf, done_buf, val_buf = [], [],
            [], [], [], []

        for _ in range(horizon):
            obs_t = torch.tensor(obs, dtype=torch.float32).unsqueeze(0)
            with torch.no_grad():
                logits, value = model(obs_t)
            dist = Categorical(logits=logits)
            action = dist.sample()

            obs_buf.append(obs_t.squeeze(0))
            act_buf.append(action)
            logp_buf.append(dist.log_prob(action))
            val_buf.append(value.squeeze())

            obs, reward, terminated, truncated, _ = env.step(action.item
                ())
            done = terminated or truncated
            rew_buf.append(reward)
            done_buf.append(done)
            episode_return += reward

```

```python
            if done:
                all_returns.append(episode_return)
                episode_return = 0.0
                obs, _ = env.reset()

        # --- compute GAE advantages ---
        with torch.no_grad():
            obs_t = torch.tensor(obs, dtype=torch.float32).unsqueeze(0)
            _, next_val = model(obs_t)
        advantages, returns = compute_gae(
            rew_buf, val_buf, done_buf, next_val.item(), gamma, gae_lambda
        )

        # --- policy updates (K epochs) ---
        obs_t   = torch.stack(obs_buf)
        act_t   = torch.stack(act_buf)
        logp_t  = torch.stack(logp_buf).detach()
        adv_t   = (advantages - advantages.mean()) / (advantages.std() + 1e-8)
        ret_t   = returns

        idx = torch.randperm(horizon)
        for _ in range(n_epochs):
            for start in range(0, horizon, batch_size):
                mb = idx[start:start + batch_size]
                logits, val_new = model(obs_t[mb])
                dist_new = Categorical(logits=logits)
                logp_new = dist_new.log_prob(act_t[mb])
                entropy  = dist_new.entropy().mean()
                ratio    = (logp_new - logp_t[mb]).exp()

                surr1 = ratio * adv_t[mb]
                surr2 = ratio.clamp(1 - clip_eps, 1 + clip_eps) * adv_t[mb]
                loss_pi  = -torch.min(surr1, surr2).mean()
                loss_vf  = ((val_new - ret_t[mb]) ** 2).mean()
                loss     = loss_pi + vf_coef * loss_vf - ent_coef * entropy

                optimizer.zero_grad()
                loss.backward()
                nn.utils.clip_grad_norm_(model.parameters(), max_grad_norm)
                optimizer.step()

        if len(all_returns) % 10 == 0:
            print(f"step={global_step}  mean_ret={np.mean(all_returns[-100:]):.1f}")

    env.close()
    return model, all_returns
```

### Group 1 — ALE/MontezumaRevenge-v5

**Key challenge:** Reward is *extremely* sparse. A standard PPO agent will rarely receive a non-zero reward signal. Students should investigate: (a) using a large entropy coefficient to promote exploration, (b) normalising rewards and advantages carefully, and (c) studying whether PPO finds *any* reward compared to DQN.

Listing 4: Group 1 starter — Montezuma's Revenge

```
# Suggested starting call for Group 1
# Increase entropy to drive exploration in sparse reward setting
model, returns = train_ppo(
    env_id      = "ALE/MontezumaRevenge-v5",
    total_steps = 5_000_000,
    horizon     = 2048,
    n_epochs    = 4,
    batch_size  = 64,
    lr          = 2.5e-4,
    gamma       = 0.99,
    gae_lambda  = 0.95,
    clip_eps    = 0.1,
    ent_coef    = 0.02,   # higher entropy for sparse exploration
    seed        = 42,
)
# NOTE: Track how often the agent enters Room 1.
# Compare with \texttt{DQN}'s first-room entry statistics from Challenge
    1.
```

### Group 2 — ALE/Pitfall-v5

**Key challenge:** Many actions cause immediate negative rewards. PPO's on-policy nature means it will unlearn risky behaviours quickly, but it may also avoid entire regions of the level. Students should investigate whether PPO discovers any positive reward, and if not, document *why* and compare failure mode with DQN's from Challenge 1.

Listing 5: Group 2 starter — Pitfall!

```
model, returns = train_ppo(
    env_id      = "ALE/Pitfall-v5",
    total_steps = 5_000_000,
    horizon     = 1024,
    n_epochs    = 6,
    batch_size  = 128,
    lr          = 1e-4,
    gamma       = 0.995,   # longer horizon helps in sparse settings
    gae_lambda  = 0.97,
    clip_eps    = 0.2,
    ent_coef    = 0.01,
    seed        = 42,
)
# NOTE: Log the fraction of episodes with positive return.
# Track the average score per episode vs. env steps.
```

### Group 3 — ALE/PrivateEye-v5

**Key challenge:** One of the hardest Atari games. Rewards are so sparse that most agents score 0 throughout training. Students should document the training collapse, compare reward histograms from PPO and DQN, and reason about whether on-policy vs. off-policy matters here.

Listing 6: Group 3 starter — Private Eye

```
model, returns = train_ppo(
    env_id      = "ALE/PrivateEye-v5",
    total_steps = 5_000_000,
    horizon     = 2048,
    n_epochs    = 4,
    batch_size  = 64,
    lr          = 2.5e-4,
    gamma       = 0.99,
    gae_lambda  = 0.95,
    clip_eps    = 0.1,
    ent_coef    = 0.02,
    seed        = 42,
)
# NOTE: Record a histogram of per-episode rewards across all runs.
# Measure and report the proportion of non-zero-reward episodes.
```

### Group 4 — ALE/Gravitar-v5

**Key challenge:** Continuous thrust and gravity require precise, sustained control. PPO's entropy bonus can maintain exploratory behaviour while the clipped ratio prevents policy collapse after a bad episode. Students should sweep entropy coefficient and horizon.

Listing 7: Group 4 starter — Gravitar

```
model, returns = train_ppo(
    env_id      = "ALE/Gravitar-v5",
    total_steps = 5_000_000,
    horizon     = 1024,
    n_epochs    = 6,
    batch_size  = 128,
    lr          = 2.5e-4,
    gamma       = 0.99,
    gae_lambda  = 0.95,
    clip_eps    = 0.2,
    ent_coef    = 0.01,
    seed        = 42,
)
# NOTE: Track the distribution of thrust actions to see if the agent
# learns to use the engine at all. Compare action histograms DQN vs. PPO.
```

### Group 5 — ALE/Solaris-v5

**Key challenge:** Multi-stage game with varying objectives. A longer horizon helps PPO capture the multi-step structures needed to navigate between stages. Students should

compare whether DQN's experience replay or PPO's longer trajectory windows work better here.

Listing 8: Group 5 starter — Solaris

```
model, returns = train_ppo(
    env_id      = "ALE/Solaris-v5",
    total_steps = 5_000_000,
    horizon     = 2048,
    n_epochs    = 4,
    batch_size  = 128,
    lr          = 2.5e-4,
    gamma       = 0.995,   # long-horizon discounting
    gae_lambda  = 0.97,
    clip_eps    = 0.2,
    ent_coef    = 0.01,
    seed        = 42,
)
# NOTE: Log whether the agent starts the second in-game stage.
# Maximum achieved in-game stage per run is a useful comparison metric.
```

### Group 6 — ALE/Venture-v5

**Key challenge:** Enemies and sparse item rewards. PPO tends to be conservative once a small positive signal is found; this may prevent further exploration. Students should investigate whether a higher entropy coefficient or a smaller clip epsilon helps PPO explore deeper into dungeons.

Listing 9: Group 6 starter — Venture

```
model, returns = train_ppo(
    env_id      = "ALE/Venture-v5",
    total_steps = 5_000_000,
    horizon     = 1024,
    n_epochs    = 6,
    batch_size  = 64,
    lr          = 1e-4,
    gamma       = 0.99,
    gae_lambda  = 0.95,
    clip_eps    = 0.1,   # more conservative updates
    ent_coef    = 0.02,
    seed        = 42,
)
# NOTE: Track how many dungeons the agent enters per run.
# Compare with \texttt{DQN}'s exploration depth from Challenge 1.
```

### Group 7 — ALE/MsPacman-v5

**Key challenge:** Denser rewards than other games in this list; this is the most tractable environment for PPO. Students should expect PPO to show clear learning curves. Focus on sample efficiency comparison: does PPO reach a score of 1000 faster or slower than DQN?

Listing 10: Group 7 starter — Ms. Pac-Man

```
model, returns = train_ppo(
    env_id      = "ALE/MsPacman-v5",
    total_steps = 5_000_000,
    horizon     = 1024,
    n_epochs    = 4,
    batch_size  = 128,
    lr          = 2.5e-4,
    gamma       = 0.99,
    gae_lambda  = 0.95,
    clip_eps    = 0.2,
    ent_coef    = 0.01,
    seed        = 42,
)
# NOTE: Sample-efficiency comparison with \texttt{DQN} is the main result here.
# Plot reward-vs-steps for both algorithms with shaded std bands.
```

### Group 8 — ALE/Phoenix-v5

**Key challenge:** Fast-moving objects require reactive policies. PPO with short horizons and many epochs should be contrasted with a longer horizon that allows credit assignment across multi-wave sequences. Students should study the effect of horizon length on PPO performance specifically.

Listing 11: Group 8 starter — Phoenix

```
model, returns = train_ppo(
    env_id      = "ALE/Phoenix-v5",
    total_steps = 5_000_000,
    horizon     = 512,    # start short for reactive tasks
    n_epochs    = 10,
    batch_size  = 128,
    lr          = 2.5e-4,
    gamma       = 0.99,
    gae_lambda  = 0.95,
    clip_eps    = 0.2,
    ent_coef    = 0.01,
    seed        = 42,
)
# NOTE: Ablate horizon lengths: 512 vs. 1024 vs. 2048.
# Report the effect on final score and training stability.
```

### Notes on scope and computational budget

PPO is typically more sample-efficient than vanilla DQN on Atari but may require more wall-clock time per environment step because it processes full rollouts before updating. Plan experiments accordingly. If compute is limited:

- Reduce `total_steps` to 2 000 000 and document this constraint clearly.
- Prioritise a 3-seed comparison between the best DQN and best PPO configuration over a wide hyperparameter sweep.

- Report trends rather than absolute best performance when budget is constrained.

### Grading checklist

The CHECKLIST.md file inside the repository must contain:

- Exact command to reproduce the best PPO run.
- Seeds used for PPO repeated experiments.
- Pointers to logs and figures for both DQN and PPO.
- A paragraph (max 200 words) summarising the key algorithmic difference observed empirically between DQN and PPO on the assigned environment.

### References and further reading

Standard starting references for this challenge include the original PPO paper by Schulman et al. (2017), the GAE paper by Schulman et al. (2016), and the DQN paper by Mnih et al. (2015). Students should also consult current survey articles on deep reinforcement learning and Atari benchmarking. All citations must follow IEEE style.

*Challenge 3 is designed to develop critical algorithmic thinking. The goal is not to make PPO beat DQN at all costs, but to understand* when, why, *and* by how much *one algorithm outperforms the other, and to express that understanding clearly in a scientific paper.*